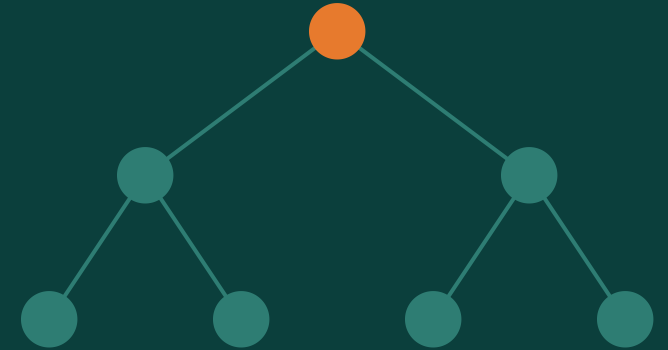




ESTRUTURAS DE DADOS E ALGORITMOS

Algoritmos de Divisão e Conquista

Aula 4 · dividir para reinar

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Qual é a complexidade de $T(n) = 5 + 4n + 3n^2$ em notação Big O?

2

Porque é que a pesquisa binária é $O(\log n)$?

3

O que acontece a uma função recursiva sem caso base?

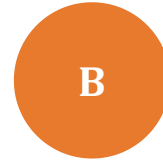
Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Tem 1000 testes para ordenar alfabeticamente e 30 minutos. Como faria?



Sozinho, um a um

Procurar o primeiro, depois o segundo...



Dividir pela turma

Cada um ordena um monte pequeno; no fim juntam-se os montes.

A opção B tem nome: divisão e conquista. Hoje vamos perceber porque é tão poderosa.

Dividir, conquistar, combinar

1

A estratégia

Os três passos e um primeiro exemplo: somar n números.

2

Analisar

A recorrência geral e o Teorema Mestre.

3

Merge Sort

O exemplo clássico: ordenar em $O(n \log n)$.

4

Praticar

Merge Sort, potência x^n e Quick Sort em Java.

Pergunta-guia: quando é que partir um problema ao meio o torna mais rápido de resolver?

1

PARTE 1

A estratégia

Um problema grande = vários problemas pequenos.

A estratégia em três passos

1

Dividir

Partir o problema em subproblemas do mesmo tipo, idealmente de tamanho semelhante.

2

Conquistar

Resolver cada subproblema — normalmente de forma recursiva. Se for pequeno o suficiente, resolve-se diretamente.

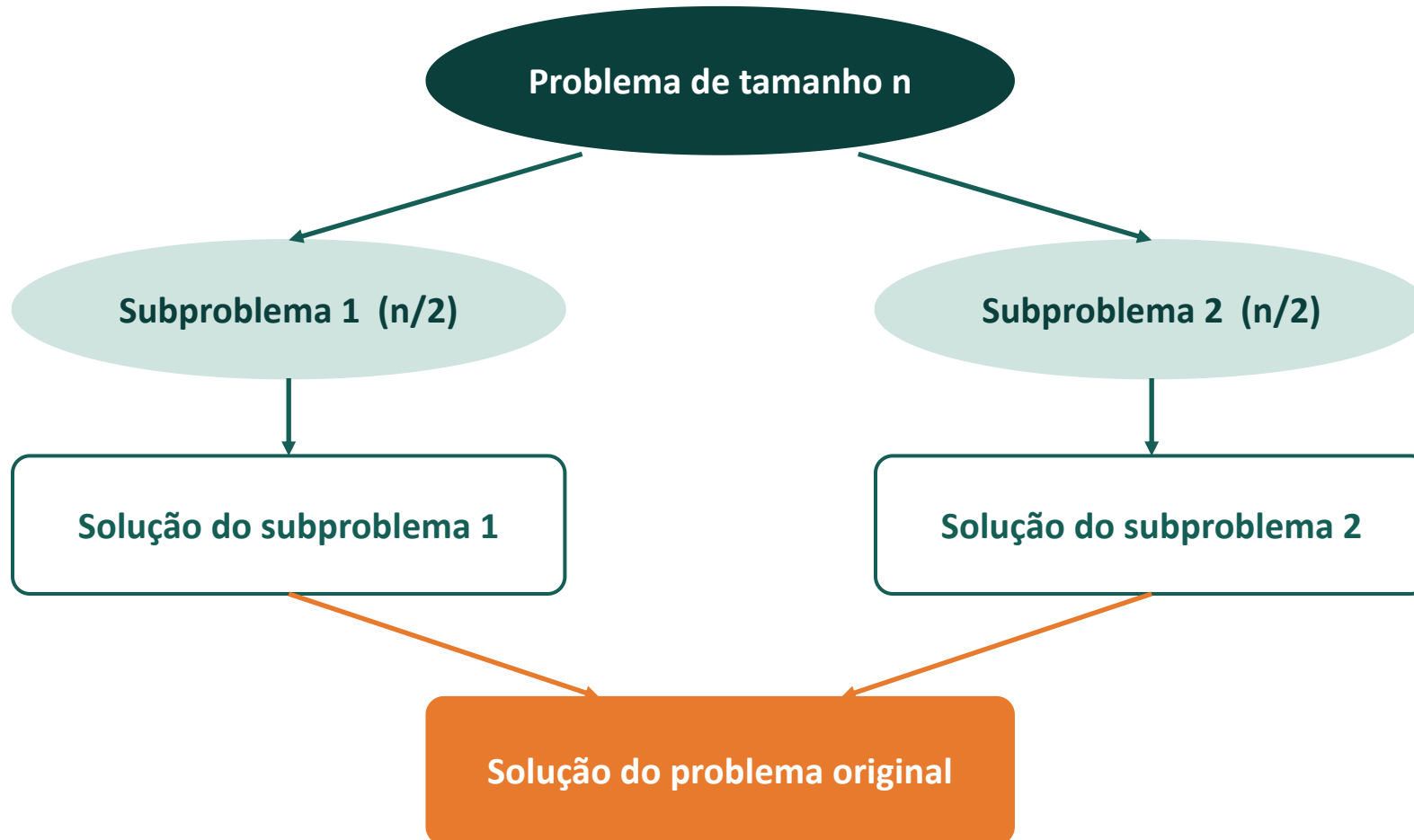
3

Combinar

Se necessário, juntar as soluções dos subproblemas para obter a solução do problema original.

Palavra-chave: recursão. Os subproblemas resolvem-se com o mesmo algoritmo, até se chegar a um caso trivial (o caso base).

O caso mais típico: dividir ao meio

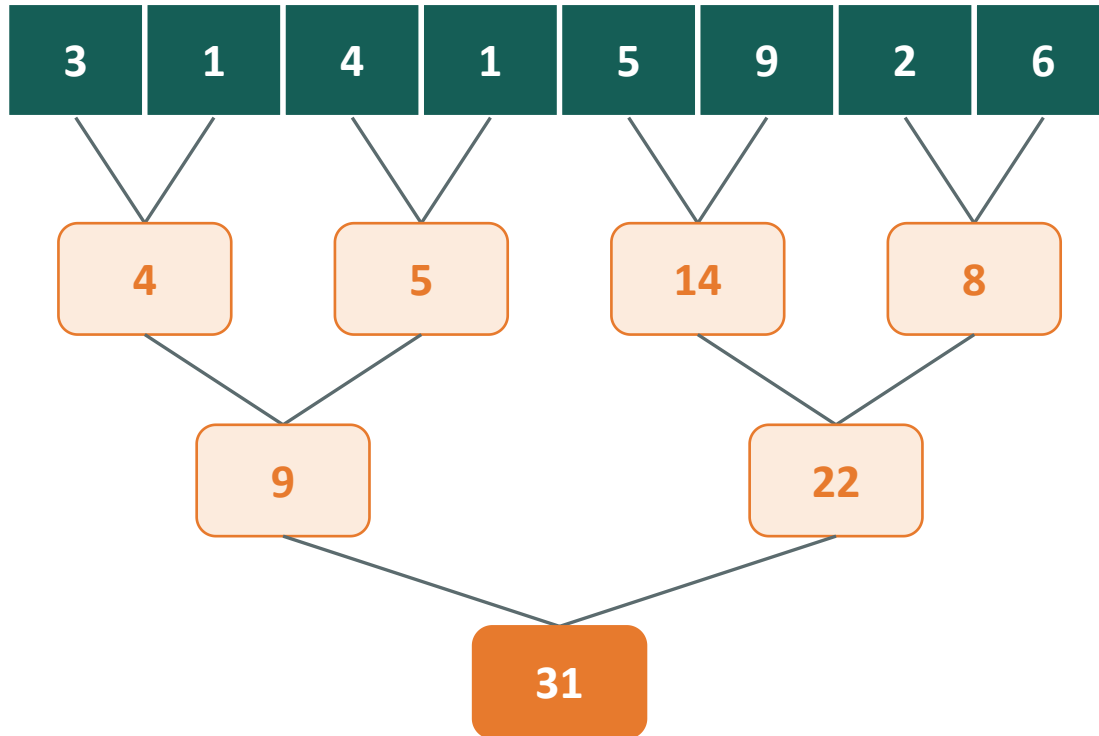


DIVIDIR

CONQUISTAR

COMBINAR

Somar n números, dividindo ao meio



somas parciais sobem na árvore

O algoritmo

Caso base: se $n = 1$, devolve a_0 .

Caso geral: soma a primeira metade, soma a segunda metade e junta os dois resultados.

$$\text{soma}(a_0 \dots a_{n-1}) =$$
$$\text{soma}(\text{primeiros } \lfloor n/2 \rfloor)$$
$$+ \text{soma}(\text{restantes } \lceil n/2 \rceil)$$

Pontos fortes e um aviso

AVISO

Nem sempre é mais eficiente do que a força bruta.

Na soma de n números, o ciclo simples e a versão recursiva fazem exatamente as mesmas $n - 1$ adições. A recursão até acrescenta o custo das chamadas.



Muitas vezes, muito mais rápido

O tempo de dividir e combinar é pequeno face ao que se poupa.



Algoritmos de referência

Merge Sort, Quick Sort, pesquisa binária, multiplicação rápida...



Ideal para computação paralela

Cada subproblema pode correr no seu próprio processador.

2

PARTE 2

Analisar

A recorrência geral e o Teorema Mestre.

Uma equação para (quase) todos

$$T(n) = a \cdot T(n / b) + f(n)$$

a

$a \geq 1$

Quantos subproblemas resolvemos

b

$b > 1$

Por quanto dividimos n (tamanho de cada: n/b)

f(n)

depende do algoritmo

Custo de dividir e de combinar as soluções

No exemplo da soma: $a = 2$, $b = 2$ e $f(n) = 1$ (uma adição para juntar as duas metades).

Resolver a recorrência sem fazer contas longas

Se $T(n) = a \cdot T(n/b) + f(n)$ com $f(n) \in \Theta(n^d)$ e $d \geq 0$, então:

CASO 1

se $a < b^d$

$$\Theta(n^d)$$

Domina o trabalho no topo (dividir e combinar).

CASO 2

se $a = b^d$

$$\Theta(n^d \log n)$$

Equilíbrio: cada nível da árvore custa o mesmo.

CASO 3

se $a > b^d$

$$\Theta(n^{\log_b a})$$

Dominam os subproblemas (as folhas da árvore).

Θ (teta) = ordem de crescimento exata. Resultados análogos valem para a notação Big O.

Aplicar o teorema em quatro passos

1 Identificar a , b e d

$$A(n) = 2 \cdot A(n/2) + 1 \quad a = 2, \quad b = 2, \quad f(n) = 1 = n^0 \rightarrow d = 0$$

2 Calcular b^d

$$2^0 = 1$$

3 Comparar a com b^d

$$2 > 1 \rightarrow \text{caso 3 } (a > b^d)$$

4 Ler o resultado

$$\Theta(n^{\log_2 2}) = \Theta(n)$$

Conclusão: tal como o ciclo simples, a soma recursiva é linear.

A SUA VEZ

A pesquisa binária (Aula 2) também é divisão e conquista:

$$T(n) = T(n/2) + 1$$

Qual é a sua complexidade, pelo Teorema Mestre?

RESPOSTA

$$a = 1, b = 2, d = 0 \rightarrow b^d = 1 = a \rightarrow \text{caso 2}$$

$$T(n) \in \Theta(n^0 \log n) = \Theta(\log n)$$

O mesmo resultado a que chegámos por intuição na Aula 2.

3

PARTE 3

Merge Sort

Ordenar por fusão: o exemplo clássico.

Ordenar por fusão em três frases

1

Dividir

Partir o array ao meio.

2

Conquistar

Ordenar cada metade —
recursivamente, com Merge Sort.

3

Combinar

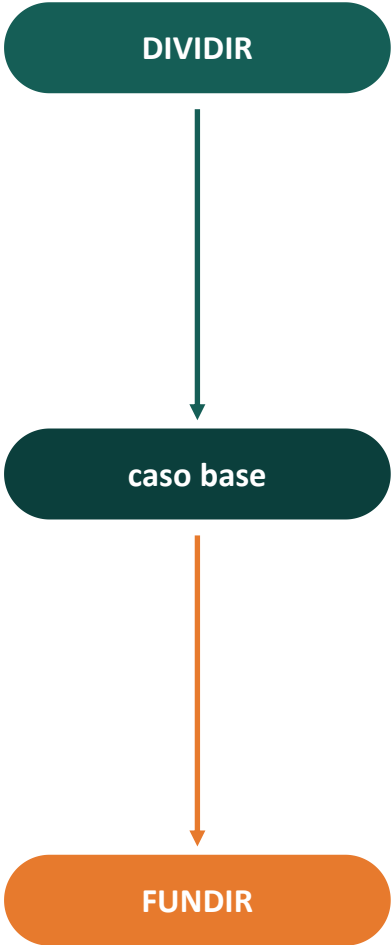
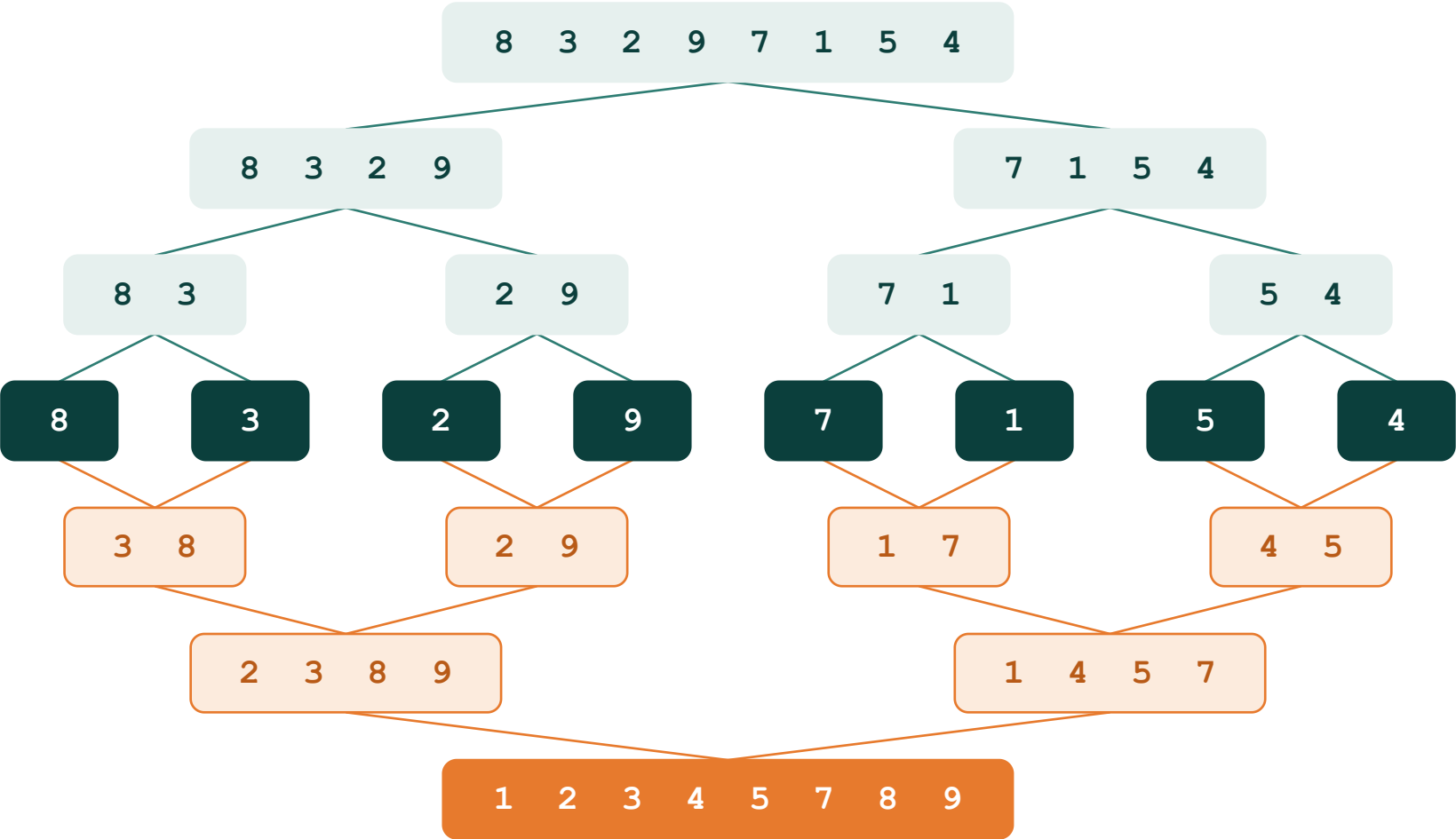
Fundir (merge) as duas metades
ordenadas numa só.

Porque é que fundir é fácil?

Imagine dois montes de cartas, cada um já ordenado. Basta comparar as duas cartas do topo e retirar sempre a menor.



Ordenar 8 3 2 9 7 1 5 4



Fundir 2 3 8 9 com 1 4 5 7

Passo	B[i] vs C[j]	Copia para A	A até agora
1	2 vs 1	1	1
2	2 vs 4	2	1 2
3	3 vs 4	3	1 2 3
4	8 vs 4	4	1 2 3 4
5	8 vs 5	5	1 2 3 4 5
6	8 vs 7	7	1 2 3 4 5 7
7	C esgotou	8 9	1 2 3 4 5 7 8 9

Quanto custa?

6

comparações neste exemplo

No pior caso: $n - 1$ comparações para fundir n elementos — ou seja, trabalho linear, $O(n)$.

Esta é a peça $f(n)$ da recorrência.

Mergesort e Merge

Mergesort(A[0..n-1])

```
se n > 1:  
  B ← A[0 .. ⌊ n/2 ⌋ - 1]  
  C ← A[⌊ n/2 ⌋ .. n-1]  
  Mergesort(B)  
  Mergesort(C)  
  Merge(B, C, A)
```

Merge(B[0..p-1], C[0..q-1], A)

```
i ← 0; j ← 0; k ← 0  
enquanto i < p e j < q:  
  se B[i] ≤ C[j]:  
    A[k] ← B[i]; i ← i + 1  
  senão:  
    A[k] ← C[j]; j ← j + 1  
  k ← k + 1  
se i = p: copiar C[j..q-1] para A[k..]  
senão: copiar B[i..p-1] para A[k..]
```

Mergesort divide e conquista (duas chamadas recursivas). **Merge** combina, em tempo linear.

Porque é que o Merge Sort é $\Theta(n \log n)$?

$$C_{\text{pior}}(n) = 2 \cdot C_{\text{pior}}(n/2) + (n - 1)$$

$$C_{\text{pior}}(1) = 0$$

$$a = 2, b = 2, d = 1 \rightarrow a = b^d \rightarrow \text{caso 2}$$

$$C_{\text{pior}}(n) \in \Theta(n \log n)$$

Ordenação $O(n^2)$

$\approx 17 \text{ min}$

$n = 1\,000\,000$

Merge Sort

$\approx 0,02 \text{ s}$

$n = 1\,000\,000$

A intuição: a árvore de recursão



$$\log_2 n \text{ níveis} \times n \text{ por nível} = n \log n$$

a 10^9 operações por segundo, como na Aula 2

Agora é consigo

1 Merge Sort

Implemente em Java o Merge Sort, com o protótipo:

```
int[] mergeSort(int[] arr)
```

Teste com 8 3 2 9 7 1 5 4 e com um array já ordenado.

2 Potência x^n

Implemente x^n por divisão e conquista (n inteiro ≥ 0):

```
power(x, 0) = 1
n par:
    power(x, n/2) * power(x, n/2)
n ímpar:
    x * power(x, n/2) * power(x, n/2)
```

Desafio: analise a sua solução com o Teorema Mestre. É mais rápida do que multiplicar x por si próprio n vezes?

3 Quick Sort

Investigue como funciona o Quick Sort e implemente-o em Java. Em que difere do Merge Sort?

Pista: escolher um pivô e partir o array em "menores" e "maiores" do que ele.

Três ideias a reter

1

Dividir, conquistar, combinar

Subproblemas do mesmo tipo, resolvidos recursivamente até ao caso base.

2

Teorema Mestre

$T(n) = aT(n/b) + f(n)$: compare a com b^d e leia o resultado.

3

Merge Sort

Fundir custa $O(n)$; há $\log n$ níveis.
Total: $\Theta(n \log n)$.

Divisão e conquista nem sempre ganha — mas, quando ganha, ganha muito.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.ª ed.). Pearson. Capítulo 5: Divide-and-Conquer.

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt